# Google Colab Source Code

Tremendous Prepaid Card Extractor

**Architecture:** Nodriver + Xvfb + System Chrome

**Target Environment:** Google Colab (Jupyter Notebook)

**Status:** 100% Working Solution

---

## 1. Colab Notebook Cell: `colab_script.py`

This is the complete script designed for Google Colab. Because Colab provides root access, it uses `apt-get` to install Xvfb and Google Chrome directly to the system path (`/usr/bin/google-chrome`). It also uses `nest_asyncio` to allow asynchronous browser automation inside Jupyter's existing event loop.

```
import os, time

# ================================================================================
# STEP 1: INSTALL DEPENDENCIES & VIRTUAL DISPLAY
# ================================================================================
print("Setting up Virtual Display (Xvfb) and Chrome...")
os.system("apt-get update -qq")
os.system("apt-get install -y -qq xvfb libvulkan1 libu2f-udev")
os.system("wget -q -O chrome.deb https://dl.google.com/linux/direct/google-chrome-
stable_current_amd64.deb")
os.system("dpkg -i chrome.deb || apt-get install -f -y -qq")
os.system("rm chrome.deb")

# Start the virtual display on port 99
os.system("Xvfb :99 -screen 0 1920x1080x24 > /dev/null 2>&1 &")
os.environ['DISPLAY'] = ':99'

os.system("pip install nodriver pandas openpyxl nest_asyncio -q")

import asyncio, re, nest_asyncio, openpyxl
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.utils import get_column_letter
from google.colab import files
import nodriver as uc

nest_asyncio.apply()

INPUT_FILENAME   = "Tremendous Payments.xlsx"
OUTPUT_FILENAME  = "Tremendous_Balances_Report.xlsx"
CHROME_PATH      = "/usr/bin/google-chrome"

# ================================================================================
# STEP 2: UPLOAD & PARSE EXCEL
# ================================================================================
```

```
print("Please upload your Excel file:")
for f in [INPUT_FILENAME, OUTPUT_FILENAME]:
    if os.path.exists(f): os.remove(f)

uploaded = files.upload()
if not uploaded: raise RuntimeError("No file uploaded.")
name = next(iter(uploaded))
if name != INPUT_FILENAME: os.rename(name, INPUT_FILENAME)

def norm_cell(v):
    if v is None: return ""
    if isinstance(v, float) and v.is_integer(): return str(int(v))
    return str(v).strip()

def get_link(cell):
    try:
        if cell.hyperlink and cell.hyperlink.target: return str(cell.hyperlink.target).strip()
    except: pass
    return None

wb_in = openpyxl.load_workbook(INPUT_FILENAME, data_only=False)
sh = wb_in["Main List"] if "Main List" in wb_in.sheetnames else wb_in.active
targets = []

for r, row in enumerate(sh.iter_rows(min_row=2, max_col=2), start=2):
    a, b = row[0], row[1]
    if a.value is None and b.value is None: continue
    rid = norm_cell(a.value) or "N/A"
    cnum = norm_cell(b.value) or "N/A"
    rurl, curl = get_link(a), get_link(b)
    if not rurl and rid.lower().startswith(("http://", "https://")): rurl = rid
    if not rurl and curl: rurl = curl
    targets.append({"row": r, "reward_id": rid, "card_number": cnum, "url": rurl, "card_url": curl})

wb_in.close()
print(f"Loaded {len(targets)} rows.")

# ================================================================================
# STEP 3: EXTRACTION LOGIC & NODRIVER
# ================================================================================
def extract_balance(text):
    if not text: return None
    t = text.replace("\u00a0", " ")
    m = re.search(r"Available\s+Balance[\s\S]{0,50}?\$\s*([\d,]+\.\d{2})", t, re.I)
    if m: return f"{float(m.group(1).replace(',', '')):.2f}"
    return None

async def start_browser():
    return await uc.start(
        browser_executable_path=CHROME_PATH,
        headless=False,
        browser_args=["--window-size=1920,1080"]
    )

async def process_url(driver, url):
    if not url: return None, "No URL"
    try:
        tab = await driver.get(url)
    except Exception as e:
        raise Exception("DriverDead")
```

```
    try:
        await tab.sleep(3)
        try:
            page_title = await tab.evaluate("document.title")
        except:
            page_title = ""

        # Cloudflare Bypass
        if "Just a moment" in page_title or "Performing security verification" in page_title:
            print(" [CF Detected] ", end="", flush=True)
            try:
                await tab.verify_cf()
                await tab.sleep(5)
            except Exception:
                pass

        # Poll for Balance
        balance = None
        for _ in range(15):
            try:
                page_text = await tab.evaluate("document.body ? document.body.innerText : ''")
                balance = extract_balance(page_text)
                if balance: break
            except: pass
            await tab.sleep(1)

        return (balance, "Success") if balance else (None, "Balance Not Found")
    except Exception as e:
        raise Exception("DriverDead")

# ==============================================================================
# STEP 4: MAIN LOOP (With Crash Recovery)
# ==============================================================================
async def main():
    results = []
    start_time = time.time()
    print("Launching Headful Browser...")
    driver = await start_browser()

    for i, t in enumerate(targets, 1):
        print(f"[{i}/{len(targets)}] {t['reward_id'][:15]}...", end="", flush=True)
        t0 = time.time()

        try:
            bal, st = await process_url(driver, t["url"])
        except Exception as e:
            # Crash Recovery: Restart browser and retry this URL
            print(f" [Browser Crashed - Restarting] ", end="", flush=True)
            try: driver.stop()
            except: pass
            await asyncio.sleep(2)
            driver = await start_browser()
            try: bal, st = await process_url(driver, t["url"])
            except: bal, st = None, "Crash Retry Failed"

        dur = time.time() - t0
        display_bal = f"${bal}" if bal else "N/A"
        print(f"-> {display_bal} ({st}) [{dur:.1f}s]")

        results.append({**t, "balance": bal or "N/A", "status": st})
        if i < len(targets): await asyncio.sleep(2)
```

```
    driver.stop()

    # ============================================================================
    # STEP 5: EXPORT EXCEL
    # ============================================================================
    wb = Workbook()
    ws = wb.active
    ws.title = "Balances"
    ws.append(["Reward ID", "Card Number", "Extracted Balance", "Overall Status"])

    header_fill = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")
    for col in range(1, 5):
        cell = ws.cell(row=1, column=col)
        cell.fill = header_fill
        cell.font = Font(bold=True, color="FFFFFF")
        cell.alignment = Alignment(horizontal="center")

    ws.freeze_panes = "A2"
    link_font = Font(color="0563C1", underline="single")

    for row_idx, item in enumerate(results, start=2):
        cell_a = ws.cell(row=row_idx, column=1, value=item["reward_id"])
        if item.get("url"): cell_a.hyperlink = item["url"]; cell_a.font = link_font

        cell_b = ws.cell(row=row_idx, column=2, value=item["card_number"])
        if item.get("card_url"): cell_b.hyperlink = item["card_url"]; cell_b.font = link_font

        ws.cell(row=row_idx, column=3, value=item["balance"])
        ws.cell(row=row_idx, column=4, value=item["status"])

    for col in ws.columns:
        max_len = max((len(str(cell.value or "")) for cell in col), default=10)
        ws.column_dimensions[get_column_letter(col[0].column)].width = min(max(max_len+4, 15), 50)

    wb.save(OUTPUT_FILENAME)

    # Summary
    elapsed = time.time() - start_time
    success_count = sum(1 for r in results if r["status"] == "Success")
    print("\n" + "="*50)
    print(f"Successful: {success_count} / {len(results)}")
    print(f"Time      : {elapsed/60:.2f} minutes")
    print("="*50)

    files.download(OUTPUT_FILENAME)
    print(f"\nDone. Report saved as {OUTPUT_FILENAME}")

# Run the async loop
await main();
```

## 2. Instructions for Google Colab

Running this script in Google Colab is very straightforward:

- Open Google Colab and create a new notebook.

- Copy the entire code block above and paste it into a single cell.
- Click the **Run** button (or press Shift + Enter).
- The script will automatically install all dependencies, download Chrome, start the virtual display, and prompt you to upload your Excel file.
- Once processing finishes, the final `.xlsx` report will automatically download to your computer.